Trabalho – Banco de Dados

Prof. Simone Dominico

Entrega: 19/11/2026
30% da nota
Defesas: 19/11 até 01/12

O trabalho poderá ser realizado individualmente ou em Trios.

1 Visão Geral

Objetivo. Implementar, em linguagem C, uma ferramenta de linha de comando (`fdtool`) para manipular Dependências Funcionais (DFs) em esquemas relacionais com atributos no intervalo `A..Z`.

A ferramenta deverá implementar:

1. cálculo do fecho X^+ de um conjunto de atributos $X \subseteq U$, dado um conjunto de dependências funcionais F;
2. cálculo da cobertura mínima de F, incluindo:
   - RHS unitário;
   - remoção de atributos estranhos;
   - remoção de DFs redundantes;
3. identificação de todas as chaves candidatas, isto é, todos os conjuntos mínimos K tais que $K^+ = U$;
4. verificação das formas normais BCNF e 3FN.

Grupo: o trabalho poderá ser realizado por **1 ou 3 alunos**.

Ambiente: execução local, sem dependências externas.

Reprodutibilidade: deverá ser fornecido um `Makefile`.

2 Formato de Entrada (`.fds`)

A entrada deverá ser fornecida em um arquivo texto. Espaços devem ser ignorados e as dependências funcionais podem ser separadas por vírgulas ou ponto e vírgula.

Exemplo:

```
U = {A,B,C,D,E}
F = { A->BC, C->D, BD->E }
```

Onde:

- U é o universo de atributos;
- F é o conjunto de dependências funcionais;
- o formato deve permitir múltiplos atributos tanto no LHS quanto no RHS.

3 Interface de Linha de Comando

O código deverá ser compilado para gerar o executável:

`fdtool`

Os seguintes subcomandos são obrigatórios.

Fecho X^+

```
fdtool closure --fds <arquivo.fds> --X <ATRIBUTOS>
```

Cobertura mínima

```
fdtool mincover --fds <arquivo.fds>
```

A saída deve possuir RHS unitário, com uma dependência por linha.

Chaves candidatas

```
fdtool keys --fds <arquivo.fds>
```

Todas as chaves candidatas devem ser apresentadas, uma por linha.

Formas normais

```
fdtool normalform --fds <arquivo.fds>
```

O programa deverá verificar BCNF e 3FN.

Saídas esperadas

- `closure`: imprime X^+ ordenado como uma string. Exemplo:

  ```
  ABCDE
  ```

- `mincover`: imprime uma DF por linha:

  ```
  A->B
  A->C
  ```

- `keys`: imprime cada chave ordenada em uma linha:

  ```
  AC
  BD
  ```

- `normalform`: deve inicialmente apresentar o status das formas normais e, caso existam violações, detalhá-las.

  Exemplo:

  ```
  BCNF: VIOLATIONS
  3NF: VIOLATIONS
  VIOLATION BCNF: B->C (B not superkey)
  VIOLATION 3NF: B->C (C not prime, B not superkey)
  ```

4 Requisitos da Implementação

A implementação deverá contemplar corretamente os conceitos de Dependências Funcionais apresentados na disciplina.

4.1 Fecho de Atributos X^+

Dado um universo de atributos U, um conjunto de dependências funcionais F e um conjunto de atributos $X \subseteq U$, o programa deverá calcular corretamente o fecho X^+ em relação a F.

O resultado deverá conter todos os atributos funcionalmente determinados por X considerando as dependências funcionais fornecidas.

4.2 Cobertura Mínima

Dado um conjunto de dependências funcionais F, o programa deverá produzir uma cobertura mínima equivalente a F.

A cobertura resultante deverá:

- ser equivalente ao conjunto original de dependências funcionais;
- possuir apenas um atributo no lado direito de cada dependência;
- não possuir atributos desnecessários nas dependências;
- não possuir dependências funcionais redundantes.

Como podem existir diferentes coberturas mínimas equivalentes, serão aceitas soluções distintas, desde que atendam às propriedades acima.

4.3 Chaves Candidatas

O programa deverá identificar **todas as chaves candidatas** do esquema relacional.

Para cada conjunto apresentado como chave candidata, deverá ser garantido que:

- o conjunto determina todos os atributos de U;
- o conjunto é mínimo, isto é, nenhum de seus subconjuntos próprios também determina todos os atributos de U.

O programa não deverá apresentar superchaves que não sejam chaves candidatas.

4.4 Verificação de BCNF e 3FN

O programa deverá verificar se o esquema satisfaz as formas normais:

- **Forma Normal de Boyce-Codd (BCNF);**
- **Terceira Forma Normal (3FN).**

A verificação deverá considerar corretamente os conceitos de:

- dependência funcional trivial e não trivial;
- superchave;
- chave candidata;
- atributo primo.

Caso uma forma normal não seja satisfeita, o programa deverá indicar as dependências funcionais responsáveis pela violação e apresentar informação suficiente para justificar a classificação.

5 Entrega

A entrega deverá conter:

1. código-fonte em C;
2. `Makefile` que gere o binário `fdtool`;
3. arquivo `README` contendo instruções de compilação, execução e exemplos;
4. arquivos de exemplo `.fds`, armazenados no diretório:

   ```
   exemplos/
   ```

5. relatório em PDF contendo:
   - definições utilizadas;
   - descrição dos algoritmos;
   - análise das complexidades;
   - três estudos de caso;
   - pelo menos um caso com violação de BCNF;
   - pelo menos um caso em 3FN, mas que não esteja em BCNF;
   - limitações da implementação;
   - principais decisões tomadas.

Os arquivos deverão ser compactados em formato:

`.tar.gz`

O nome do arquivo deverá conter o nome e o RA dos alunos.

Exemplos:

```
simone_23456_maria_34567_jose_891011.tar.gz
```

6 Avaliação

A atividade vale **30 pontos**.

Critério	**Pontos**
Fecho X^+ correto e robusto	6
Cobertura mínima correta	6
Chaves candidatas e minimalidade	6
Verificação BCNF/3FN	8
Relatório e organização	4
Total	**30**

7 Exemplos de Uso

Fecho

Considere:

$$U = \{A, B, C, D, E\}$$

e

$$F = \{A \to BC,\ C \to D,\ BD \to E\}.$$

```
$ fdtool closure --fds exemplos/e1.fds --X A
ABCDE

$ fdtool closure --fds exemplos/e1.fds --X BD
BDE
```

Cobertura mínima

```
$ fdtool mincover --fds exemplos/e1.fds
A->B
A->C
C->D
BD->E
```

Chaves

```
$ fdtool keys --fds exemplos/e2.fds
AC
```

Formas normais

Considere:

$$U = \{A, B, C\}$$

e

$$F = \{A \to B,\ B \to C\}.$$

```
$ fdtool normalform --fds exemplos/e3.fds
BCNF: VIOLATIONS
3NF: VIOLATIONS
VIOLATION BCNF: B->C (B not superkey)
VIOLATION 3NF: B->C (C not prime, B not superkey)
```

8 Política e Integridade

- O trabalho poderá ser realizado **em trios**.
- **Os três alunos são responsáveis por todo o código e devem ser capazes de explicar todas as decisões e partes da implementação**.
- O código deverá ser autoral.
- Referências externas utilizadas deverão ser citadas. Uso de IA deve ser citada.
- O programa deverá compilar utilizando:

  ```
  gcc -std=c11 -Wall -Wextra
  ```

- Os resultados deverão ser reprodutíveis utilizando os exemplos fornecidos.

- Os resultados devem ser idênticos a saída de exemplo, caso contrário a nota será reduzida em 30%.